Enhancing Hand-Tracking UX/UI Interactions in Extended Reality Documentation

Unity Setup

Download Unity Hub and click Install Editor in the Installs menu. Select Unity 6.2 (6000.2.14f1) and make sure to have Android Build Support selected. After everything is installed, click New Project and have Core Universal 3D selected. Name your project and click Create Project.

In the asset store, add Meta XR All-in-One SDK to your account. This enables a way to have hand-tracking and UI interactions in Unity. Go to Windows → Package Management → Package Manager. Select My Assets and download Meta XR All-in-One SDK. In addition, click the samples tab in the same menu and import example scenes.

Now go to the new tab called Meta XR Tools and use the project setup tool to fix any issues by pressing a button next to the issue, or follow the instructions if that button isn't available.

Setting Up Scenes

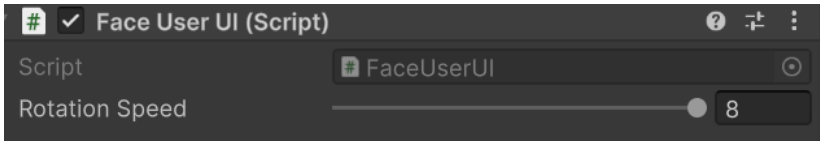
For a basic scene, the UI (design) was grabbed from the example scenes (Meta XR Interaction SDK → Example Scenes. The comprehensive Rig Example scene was used to edit and implement new features. It was also a way to compare Meta's features to the new interactions as closely as possible. The menu floating in the middle of the table was used.

The basic layout of the UI was edited. The middle rotators, top and bottom, were removed. Remove the extra collider on each edge and cover the entire edge with the one leftover collider. The colliders were also reduced to accurately represent the size of the Meta's menus (about 0.03 in width and 0.35 in height). The settings for the rest will change for each scene (feature).

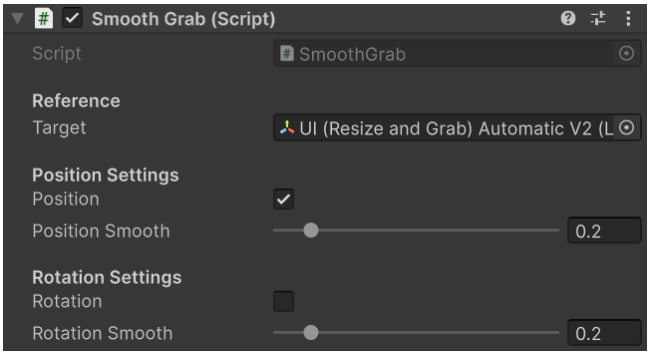
Automatic Scene

This feature will allow the UI to always face the user. Since this feature is not available in the example scene, it had to be manually implemented.

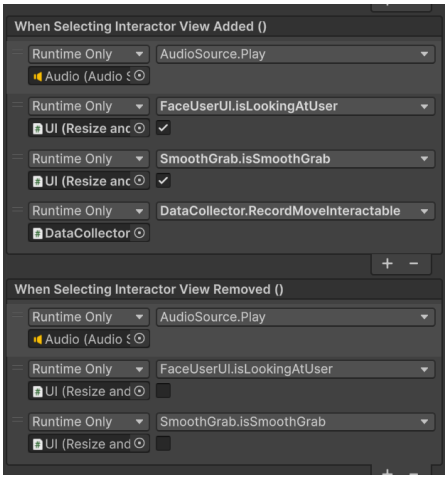
The **Face User UI** script was implemented to have the UI face the user when grabbing the UI. It has a rotation speed to determine how fast the UI needs to face the user.



To make the UI less shaky when moving, the **Smooth Grab** script was implemented. Since the position only needs to be smoothed, the position was checked. The rotation is only checked in the manual scene to smooth the rotation as well.



To make the Face User UI and Smooth Grab work when moving the UI, interaction events must be set.



Go to Interactables → PanelWithManipulators_UIExample → PanelInteractable. Under the inspector, open up **Interactable Unity Event Wrapper**.

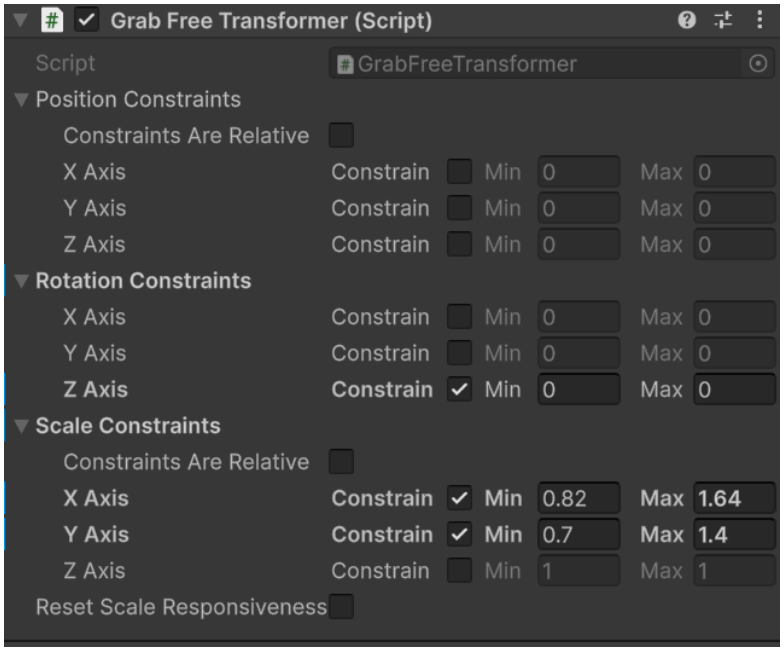
Under the **Interactable Unity Event Wrapper**, go to **When Selecting Interactor View Added/Removed**. Press the plus (+) button to add an event for both the Added and Removed event lists. This is where the Face User UI and Smooth Grab scripts are dragged and dropped.

In the drop-down menu, select **isLookingAtUser** for **FaceUserUI** and **isSmoothGrab** for **SmoothGrab** to all events. Uncheck both of them in the Removed section.

The Face User UI and Smooth Grab script must be placed on the PanelWithManipulators_UIExample object. This scene will be used to compare with the manual scene.

Manual Scene

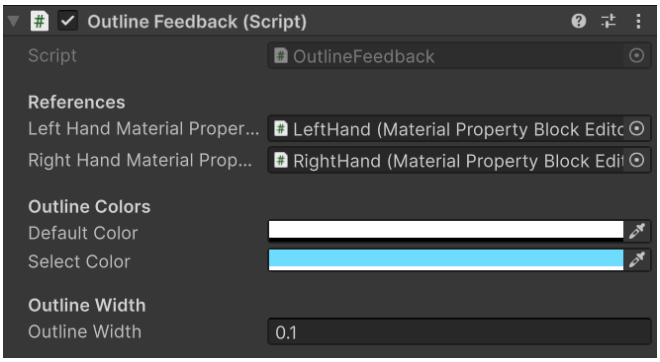
The manual scene is basically the same setup as the Automatic Scene, so duplicating the scene is recommended. Some differences are that the smooth grab rotation setting is active, and one axis is disabled. The important difference is the addition of the Grab Free Transformer under the PanelWithManipulators_UIExample object. The Z axis will be set at 0 constantly to prevent the UI from rotating sideways. Copy the settings below:



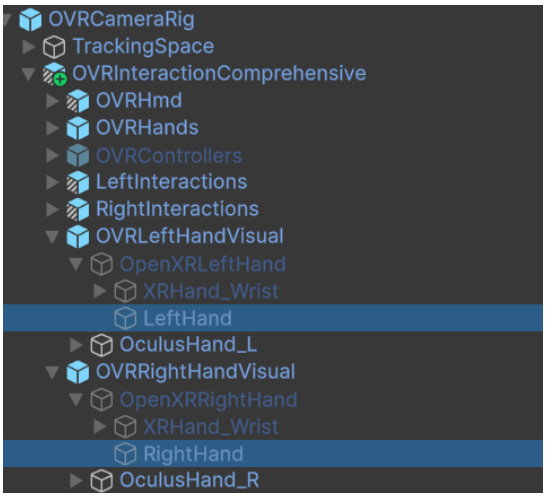
Remove the **Face User UI** script to not have the UI face the user.

Outline Scene

This scene uses the automatic scene as the main base, so it is recommended to duplicate it. An Outline Feedback script was implemented for the hand outline functionality. Developers will be able to change the outline colors and the width as well.



To make this work, the left and right-hand **material property block editor** must be referenced for both left and right hands. Afterwards, the color and float properties must be set to enable this to work. The setting is located under:

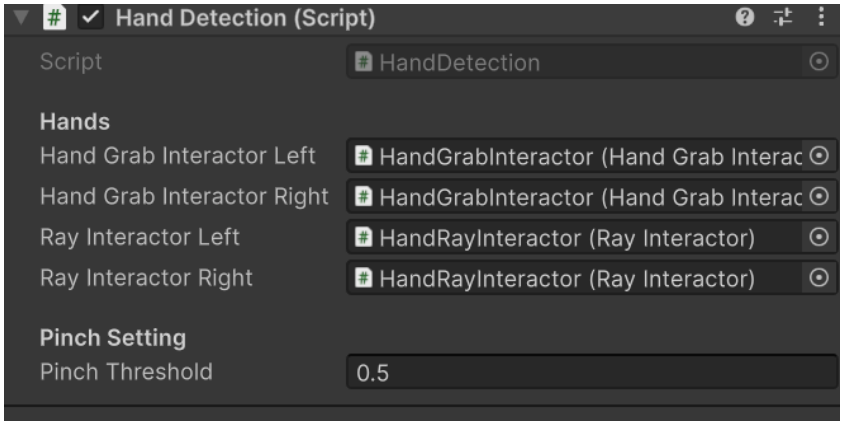


Left Hand:
OVRCameraRig → OVRInteractionComprehensive → OVRLeftHandVisual → OpenXRLeftHand → LeftHand

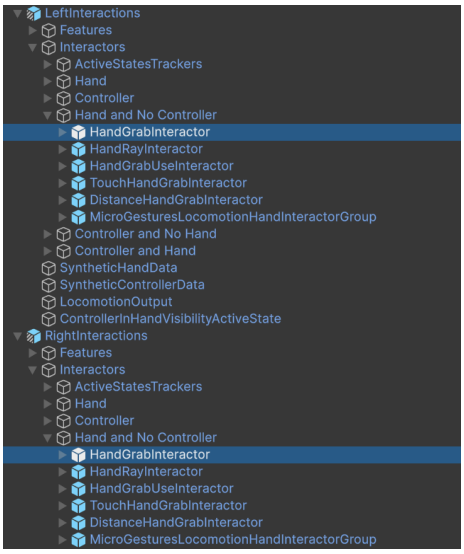
Right Hand:
OVRCameraRig → OVRInteractionComprehensive → OVRRightHandVisual → OpenXRRightHand → RightHand

In the inspector, the Color Properties and Float Properties are empty. Add them by pressing the plus (+) sign. For the color, name it **_OutlineColor** and set it to black. For the float, name it **_OutlineWidth** and set it to 0.001.

To know when the outline should appear (interacting with the resize or move zone), a Hand Detection script was implemented.



For the Hand Grab and Ray Interactors, they are located at:



Left Hand:
OVRCameraRig → OVRInteractionComprehensive → LeftInteractions→ Hand and No Controller→ HandGrabInteractor

Right Hand:
OVRCameraRig → OVRInteractionComprehensive → RightInteractions → Hand and No Controller→ HandGrabInteractor

The **Ray Interactors** are located right below the HandGrabInteractor object.

Both the **Outline Feedback** and **Detection Scripts** can be placed on any empty objects.

Note: The Automatic, Manual, Regular Interaction, and Updated Interaction scenes use these outline features.

No Outline Scene

The outline scene was duplicated to make the no outline scene. The Outline Feedback and Hand Detection scripts were disabled. It will be used as a comparison to the outline scene.

Regular Interaction Scene

The outline scene was duplicated to make the regular interaction scene. The difference is that the size of the collider for all sides is smaller (0.3 scale for width and height), and the corner visual feedback is removed to match the regular corner scale grab in the meta menu.

Updated Interaction Scene

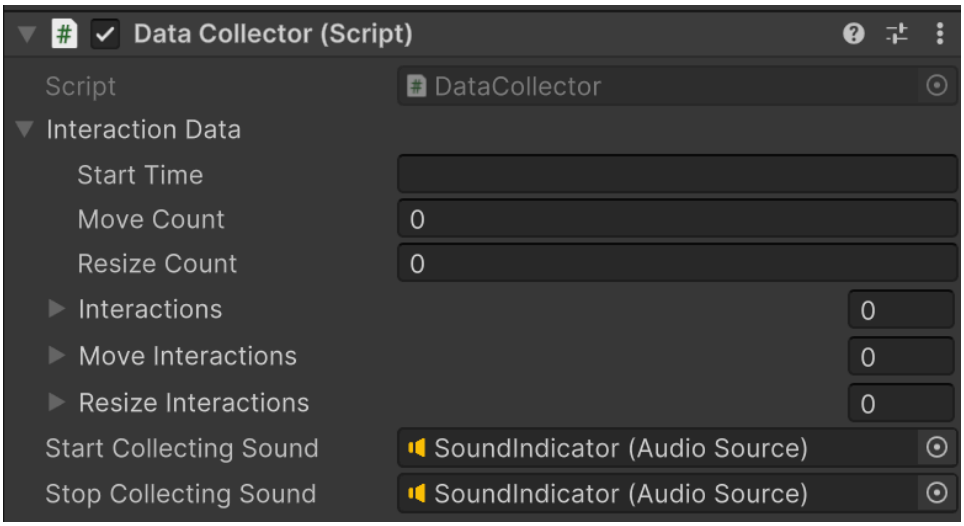
The Updated Interaction Scene is the same as the automatic scene to compare with the regular interaction scene.

Data Collection

Data were collected in this project. They collect the number of times the participants interact with the move and resize interaction zones and the timestamps for each interaction. The start time was also included to differentiate between participants.

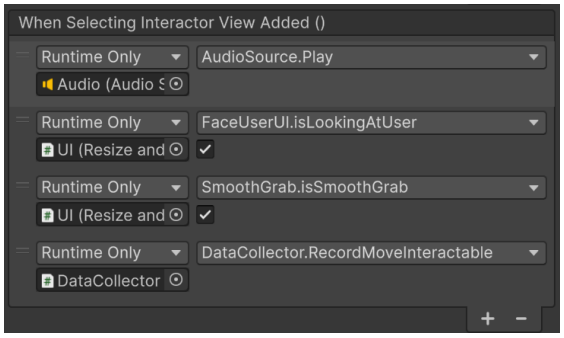
Data Collector

Is a way for the developers to start and end the data collection. When the developer presses the A button on the controller, the data collection begins. If the developer wants to stop the data collection, they can press the B button on the controller.



When the developer presses A, it will play a sound that indicates that the data collection has begun. This is the same for when the developer presses the B button on the controller.

To collect the data, events were used to record the move.



Go to Interactables → PanelWithManipulators_UIExample → PanelInteractable. Under the inspector, open up **Interactable Unity Event Wrapper**.

Add the data collector script to the list of events in “When Selecting Interactor View Added” and select RecordMoveInteractable in the drop-down menu.

For the resize, select the resize interaction colliders on the UI. In each of the colliders, add the data collector script to the list of events in “When Selecting Interactor View Added” and select RecordResizeInteractable in the drop-down menu. This should be similar to the process above.

Data Classes

The data classes script contains variables and lists that will be stored locally. This includes startTime, move count, resize count, list of interactions, list of move interactions, and list of resize interactions.

Each list, it contains Interaction elements that will store interaction times.

Data Manager

The data manager script will save the data by calling the SaveData method. From the data collector script, when the B button is pressed on the controller, it will save the data to the local file. It uses persistentDataPath, so it will be saved under Android/data/<package_name> on the Oculus Quest.